Harmonic and Computational Analysis of Odd Numbers via Truncated Dirichlet Series: Exploration of Five Algorithmic Configurations

Abstract

This study presents a novel computational and harmonic approach to analyzing the distribution of odd numbers using truncated Dirichlet series on the critical line ($\sigma = 1/2$). Building on a discrete phase operator parameterized by a polynomial equation for s_2, we explore five distinct algorithmic configurations. These combine a vectorized, exhaustive sweep over base pairs (k_a, k^b) to detect ratio invariants under a positivity/negativity criterion on the real part of a product, together with targeted evaluations on fixed base pairs ((5, 7), (10, 14), and (4, 14)) across training and out-of-sample validation samples. The goal is to examine whether these harmonic resonances can capture hidden structures related to primality.

Introduction

The search for links between the spectral properties of zeta functions (or analogous harmonic series) and the distribution of prime numbers has long motivated the development of complex analytic methods. In this spirit, the approaches explored here focus on constructing an oscillatory signal driven by a spectral parameter b_t, derived from the arcsine of a polynomial root s_2.

Rather than using conventional primality tests, our approach examines the real part of the product of two harmonic sums evaluated before and after an odd candidate n (evaluated at n−1 and n+1). This work synthesizes the evolution of five main algorithms:

1. The extended vectorized sweep algorithm (n ≤ 100, KA_KB_MAX = 20), optimized via NumPy to identify (k_a, k^b) pairs satisfying the negative-real-part product criterion and to analyze possible recurring k_a / k^b ratios.
2. The initial pilot algorithm, laying the groundwork for the combinatorial exploration for n in [5, 21].
3. The fixed-base algorithms (A, B, C), respectively testing the configurations (5, 7), (10, 14), and (4, 14) on a training set and validated over an extended range (n from 101 to 399).

The Key Algorithms

The five algorithms have been extracted into separate Python files accompanying this document:
1. algo1_vectorized_extended_sweep.py — Extended vectorized sweep (n ≤ 100, KA_KB_MAX = 20)
2. algo2_initial_pilot.py — Initial pilot algorithm (n in [5, 21])
3. algo3_fixed_base_A_5_7.py — Fixed base pair (5, 7)
4. algo4_fixed_base_B_10_14.py — Fixed base pair (10, 14)
5. algo5_fixed_base_C_4_14.py — Fixed base pair (4, 14)

Key Algorithms:

1》

```python
```

```
import numpy as np
from scipy.optimize import brentq
from collections import defaultdict

def _equation(s2, k3):
    A = 8*s2**6 - 24*s2**4 + 24*s2**2 - 8
    return abs(A)**(2/3)*(24*k3**2*s2**6 - 72*k3**2*s2**4 - 24*k3**2) + 4*s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

t = 1
s2 = _get_s2(t)
kmin, kmax = -1500, 1500
signe = 1

# --- Precalcul du vecteur s = 1/2 + i*b_t (fait UNE SEULE FOIS, pas a chaque appel) ---
k_vals = np.array([k for k in range(kmin, kmax + 1) if k != 0])
b_t = (np.arcsin(signe * abs(s2)) - 2 * k_vals * np.pi) / np.log(2)
S_VEC = 0.5 + 1j * b_t   # vecteur numpy, reutilise partout

def S_val(ka, kb, test_n):
    # version vectorisee : plus de boucle Python, numpy fait la somme
    return np.sum(ka**(-S_VEC) + kb**(-S_VEC) + test_n**(-S_VEC))

solutions_by_n = {}

# --- Plage etendue : n jusqu'a 100 ---
n_list = range(5, 101, 2)

# --- IMPORTANT : plafonner ka,kb a une valeur fixe (pas "< n") ---
# Sinon le nombre de paires explose (C(n-2,2) pour n=99 = ~4700 paires)
KA_KB_MAX = 20   # a ajuster : 20 donne C(18,2)=153 paires par n, gerable

print("=== 1. Recherche des paires (ka, kb) par valeur de n ===")
for n in n_list:
    valid_pairs = []
    is_prime = all(n % i != 0 for i in range(2, int(np.sqrt(n)) + 1))

    borne = min(KA_KB_MAX, n)  # ne jamais depasser n de toute facon
    for ka in range(2, borne):
        for kb in range(ka + 1, borne):
            S_avant = S_val(ka, kb, n - 1)
            S_apres = S_val(ka, kb, n + 1)
            produit = S_avant * S_apres

            if produit.real < 0:
                valid_pairs.append((ka, kb))

    solutions_by_n[n] = valid_pairs
    statut_str = "Premier" if is_prime else "Compose"
    print(f"n = {n:3d} ({statut_str:7s}) -> {len(valid_pairs)} paires valides")
```

```
print("\n=== 2. Analyse des repetitions et des rapports (ka / kb) inter-n ===")
ratio_occurrences = defaultdict(list)
for n, pairs in solutions_by_n.items():
    for ka, kb in pairs:
        ratio = round(ka / kb, 4)
        ratio_occurrences[ratio].append((n, ka, kb))

found_repetitions = False
for ratio, occurrences in sorted(ratio_occurrences.items()):
    if len(occurrences) > 1:
        found_repetitions = True
        print(f"{ratio:<10} | {occurrences}")

if not found_repetitions:
    print("Aucune repetition trouvee.")
```

From :

```
import numpy as np
from scipy.optimize import brentq
from collections import defaultdict

def _equation(s2, k3):
    A = 8*s2**6 - 24*s2**4 + 24*s2**2 - 8
    return abs(A)**(2/3)*(24*k3**2*s2**6 - 72*k3**2*s2**4 - 24*k3**2) + 4*s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

t = 1
s2 = _get_s2(t)          # ≈ -0.99941091795
kmin, kmax = -1500, 1500 # Réduit légèrement pour optimiser le temps de calcul du balayage
signe = 1

def S_val(ka, kb, test_n):
    total = 0
    for idx_kb in range(kmin, kmax + 1):
        if idx_kb != 0:
            b_t = (np.arcsin(signe * abs(s2)) - 2 * idx_kb * np.pi) / np.log(2)
            s = 0.5 + 1j * b_t
            total += 1/(ka**s) + 1/(kb**s) + 1/(test_n**s)
    return total

# Dictionnaire pour stocker les paires validées pour chaque n
solutions_by_n = {}

# Plage de test pour n (impairs)
n_list = range(5, 21, 2)

print("=== 1. Recherche des paires (ka, kb) par valeur de n ===")
```

```
for n in n_list:
    valid_pairs = []
    is_prime = all(n % i != 0 for i in range(2, int(np.sqrt(n)) + 1))

    # On fait varier ka et kb inférieurs à n (avec ka < kb pour éviter les doublons symétriques)
    for ka in range(2, n):
        for kb in range(ka + 1, n):
            S_avant = S_val(ka, kb, n - 1)
            S_apres = S_val(ka, kb, n + 1)
            produit = S_avant * S_apres

            # Critère de sélection : partie réelle du produit négative
            if produit.real < 0:
                valid_pairs.append((ka, kb))

    solutions_by_n[n] = valid_pairs
    statut_str = "Premier" if is_prime else "Composé"
    print(f"n = {n:2d} ({statut_str:7s}) -> {len(valid_pairs)} paires valides trouvées : {valid_pairs}")

print("\n=== 2. Analyse des répétitions et des rapports (ka / kb) inter-n ===")
ratio_occurrences = defaultdict(list)

for n, pairs in solutions_by_n.items():
    for ka, kb in pairs:
        # On calcule le rapport arrondi à 4 décimales pour observer les proximités
        ratio = round(ka / kb, 4)
        ratio_occurrences[ratio].append((n, ka, kb))

# Affichage des rapports qui apparaissent pour plusieurs n ou plusieurs paires
print(f"{'Rapport (ka/kb)':<15} | {'Occurrences (n, ka, kb)'}")
print("-" * 65)
found_repetitions = False
for ratio, occurrences in sorted(ratio_occurrences.items()):
    if len(occurrences) > 1:
        found_repetitions = True
        print(f"{ratio:<15} | {occurrences}")

if not found_repetitions:
    print("Aucun rapport ka/kb strictement identique n'a croisé les différents n sur cette plage pilote.")
```

2》

(fixed base, tested on the 5–95 training sample and then on the 101–399 validation sample). Simply change `KA, KB` at the top of the script and adjust `N_TRAIN_MAX` / `N_VAL_MIN, N_VAL_MAX` to extend the ranges.

Algorithme A — paire (5, 7)

```python
```

```
import numpy as np
from scipy.optimize import brentq

def _equation(s2, k3):
    A = 8*s2**6 - 24*s2**4 + 24*s2**2 - 8
    return abs(A)**(2/3)*(24*k3**2*s2**6 - 72*k3**2*s2**4 - 24*k3**2) + 4*s2**4

def _get_s2(t):
    return brentq(_equation, -0.9999, -0.0001, args=(-t,), xtol=1e-14)

def sieve(N):
    isp = [True]*(N+1); isp[0]=isp[1]=False
    for i in range(2, int(N**0.5)+1):
        if isp[i]:
            for j in range(i*i, N+1, i): isp[j]=False
    return isp

# ---- Parametres de la base a tester ----
KA, KB = 5, 7

t = 1
s2 = _get_s2(t)          # ~= -0.99941091795
kmin, kmax = -800, 800   # taille du reseau (augmenter pour plus de precision, plus lent)
signe = 1

def S_val(ka, kb, test_n):
    total = 0
    for k_b in range(kmin, kmax + 1):
        if k_b != 0:
            b_t = (np.arcsin(signe * abs(s2)) - 2*k_b*np.pi) / np.log(2)
            s = 0.5 + 1j*b_t
            total += 1/(ka**s) + 1/(kb**s) + 1/(test_n**s)
    return total

def predict_prime(ka, kb, n):
    S_avant = S_val(ka, kb, n - 1)
    S_apres = S_val(ka, kb, n + 1)
    return (S_avant * S_apres).real < 0

# ---- Plages a etendre ----
N_TRAIN_MAX = 95     # entrainement : n = 5..N_TRAIN_MAX (impairs)
N_VAL_MIN, N_VAL_MAX = 101, 399   # validation hors-echantillon

is_p = sieve(N_VAL_MAX + 5)

def evaluate(n_list, label):
    correct = 0
    for n in n_list:
        if predict_prime(KA, KB, n) == bool(is_p[n]):
            correct += 1
    acc = correct/len(n_list)
    n_premiers = sum(is_p[n] for n in n_list)
    baseline = max(n_premiers, len(n_list)-n_premiers) / len(n_list)
```

```
    print(f"{label}: precision = {correct}/{len(n_list)} = {acc:.1%}   (baseline triviale = {baseline:.1%})")
    return acc

Ns_train = list(range(5, N_TRAIN_MAX + 1, 2))
Ns_val   = list(range(N_VAL_MIN, N_VAL_MAX + 1, 2))

print(f"=== Paire (ka={KA}, kb={KB}) ===")
evaluate(Ns_train, "Entrainement")
evaluate(Ns_val,   "Validation")
```

Algorithm B — pair (10, 14)

Identical; only the following line changes:
```python
KA, KB = 10, 14
```

Algorithm C — pair (4, 14)

Identical; only the following line changes:
```python
KA, KB = 4, 14
```

Discussion — Existing Primality-Sorting Formulas and Potential Efficiency Gains

No simple, fast, and universally efficient formula exists to generate or sort (separate) all prime numbers directly. Nevertheless, several exact or theoretical formulas have been discovered over the course of mathematical history. They fall broadly into structural filters, exact-but-impractical formulas, generating polynomials, and non-constructive existence theorems. Reviewing them is useful here because several could, in principle, be combined with the pilot algorithm (algo2_initial_pilot.py) to reduce its computational cost.

Structural filters: the 6n ± 1 form

Every prime greater than 3 can be written as 6n + 1 or 6n − 1 for some positive integer n. For n = 2, for instance, 6(2) − 1 = 11 and 6(2) + 1 = 13. The converse does not hold — 6(4) + 1 = 25 is composite — so the filter does not identify primes on its own, but it immediately discards two-thirds of all integers as candidates, which makes it an efficient first-pass sieve.

Exact formulas that are impractical at scale

Wilson's theorem states that an integer n > 1 is prime if and only if (n − 1)! + 1 is divisible by n. This yields a sorting function f(n) = 2 + (2·n! mod (n+1)): the formula returns n + 1 when n + 1 is prime, and 2 otherwise. In practice, computing a factorial becomes astronomically expensive as n grows, which rules the formula out for any but the smallest candidates. Willans' formula (1964) similarly gives the n-th prime explicitly, combining trigonometric functions, floor functions, and factorials — it is mathematically exact but purely theoretical, with no practical computational value.

Generating polynomials

Euler's polynomial f(n) = n² + n + 41 produces a prime for every integer n from 0 to 39, but fails at n = 40, where $40^2 + 40 + 41 = 41^2$ is composite. No simple polynomial is known to generate only primes for all n. The Jones polynomial goes further: a system of polynomials in 26 variables whose set of positive values coincides exactly

with the primes. Its limitation is that it produces negative numbers the vast majority of the time, making it unusable as a practical sorting tool.

The Sieve of Eratosthenes

The Sieve of Eratosthenes is not an algebraic formula but a historical filtering algorithm: it eliminates successive multiples of each prime found. It remains one of the most practical exact methods for enumerating primes up to a bound, though it becomes memory-intensive for very large ranges.

The Green–Tao theorem and asymptotic approaches

Rather than offering a formula to sort or isolate primes, the Green–Tao theorem (2004) proves that the sequence of primes contains arithmetic progressions of arbitrary length — a progression being a sequence of primes separated by a constant gap (for example 3, 5, 7 with gap 2, or 5, 11, 17, 23, 29 with gap 6). The theorem is non-constructive: it guarantees that progressions of any length, however long, must exist somewhere along the number line, without providing any means of computing or locating them. More broadly, Terence Tao's work treats the primes not as a sequence fixed by a formula but through the lens of additive combinatorics and pseudorandomness, using asymptotic formulas that predict the density or likelihood of prime structures in an interval rather than their exact position.

Implications for the pilot algorithm's efficiency

algo2_initial_pilot.py currently determines the reference primality label for each n via trial division up to $\sqrt{n}$, and separately evaluates every (ka, kb) pair through the costly harmonic sum S_val, which loops over roughly 3000 values of k_b in pure Python. The reference-labeling step is not the bottleneck, but it can still be tightened: replacing the plain trial-division loop with a $6n \pm 1$ wheel (skipping multiples of 2 and 3 outright, then testing only candidates of the form $6k \pm 1$) discards a large fraction of divisors to check without changing the result. This has been added to the accompanying script as an is_prime_fast helper, used as a drop-in replacement for the reference is_prime computation.

None of the exact formulas above (Wilson, Willans, Jones) are viable replacements for the harmonic resonance test itself, since their own computational cost grows faster than the direct approach they replace. Their main value here is conceptual: they illustrate that closed-form primality criteria exist but are traded against tractability, which is precisely the trade-off the resonance-based approach in this study is also navigating. The Sieve of Eratosthenes remains the most practical classical option if the goal shifts from testing individual candidates to labeling an entire range at once, and could be used to precompute reference labels for the whole n_list in a single pass rather than re-deriving primality for each n independently.

Conclusion

The implementation and comparative evaluation of these five algorithmic configurations demonstrate the complexity of the interaction between harmonic phase interference and the multiplicative structure of integers.

On one hand, the vectorized exhaustive-sweep approach makes it possible to efficiently structure the search for pairs under controlled complexity constraints, opening the way to analyzing equivalence classes of ratios. On the other hand, the tests on fixed bases (such as (5, 7), (10, 14), and (4, 14)) reveal the nature of the learning and out-of-sample validation dynamics of these harmonic models.

In sum, while these architectures highlight powerful diophantine resonance phenomena, they also underscore the need to refine the selection criteria in order to distinguish pure interference noise from genuine signatures of primality.

Appendix — Pilot Algorithm Output (n = 5 to 19)

1. Search for (ka, kb) pairs by value of n

```
n =  5 (Prime) -> 2 valid pairs found: [(2, 3), (2, 4)]
n =  7 (Prime) -> 7 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (3, 5), (3, 6), (5, 6)]
n =  9 (Composite) -> 12 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (3,
5), (3, 6), (3, 7), (5, 6), (5, 7), (6, 7)]
n = 11 (Prime) -> 11 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9),
(2, 10), (7, 9), (7, 10), (9, 10)]
n = 13 (Prime) -> 24 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9),
(2, 10), (2, 11), (2, 12), (3, 5), (3, 7), (3, 11), (5, 12), (7, 9), (7, 10), (7, 11), (7, 12), (9,
10), (9, 11), (9, 12), (10, 11), (10, 12), (11, 12)]
n = 15 (Composite) -> 31 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2,
9), (2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3, 10),
(3, 11), (3, 12), (3, 13), (3, 14), (5, 8), (6, 8), (7, 8), (8, 9), (8, 10), (8, 11), (8, 12), (8,
13), (8, 14)]
n = 17 (Prime) -> 45 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9),
(2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (2, 15), (3, 5), (3, 6), (3, 7), (3, 8), (3, 9), (3,
10), (3, 11), (3, 12), (3, 13), (3, 14), (3, 16), (5, 8), (5, 16), (6, 8), (6, 16), (7, 8), (7, 16),
(8, 9), (8, 10), (8, 11), (8, 12), (8, 13), (8, 14), (8, 15), (8, 16), (9, 16), (10, 16), (11, 16),
(12, 16), (13, 16), (14, 16), (15, 16)]
n = 19 (Prime) -> 87 valid pairs found: [(2, 3), (2, 4), (2, 5), (2, 6), (2, 7), (2, 8), (2, 9),
(2, 10), (2, 11), (2, 12), (2, 13), (2, 14), (2, 15), (2, 16), (2, 17), (2, 18), (3, 5), (3, 7),
(3, 10), (3, 11), (3, 13), (3, 14), (3, 16), (3, 17), (3, 18), (5, 12), (5, 13), (5, 14), (5, 15),
(5, 16), (5, 17), (5, 18), (6, 15), (6, 16), (7, 9), (7, 10), (7, 11), (7, 12), (7, 13), (7, 14),
(7, 15), (7, 16), (7, 17), (7, 18), (8, 16), (9, 10), (9, 11), (9, 12), (9, 13), (9, 14), (9, 15),
(9, 16), (9, 18), (10, 11), (10, 12), (10, 13), (10, 14), (10, 15), (10, 16), (10, 17), (10, 18),
(11, 12), (11, 13), (11, 14), (11, 15), (11, 16), (11, 17), (11, 18), (12, 13), (12, 14), (12, 15),
(12, 16), (12, 18), (13, 14), (13, 15), (13, 16), (13, 18), (14, 15), (14, 16), (14, 17), (14, 18),
(15, 16), (15, 17), (15, 18), (16, 17), (16, 18), (17, 18)]
```

2. Analysis of repetitions and (ka / kb) ratios across n

```
Ratio (ka/kb)    | Occurrences (n, ka, kb)
-----------------------------------------------------------------
0.1333           | [(17, 2, 15), (19, 2, 15)]
0.1429           | [(15, 2, 14), (17, 2, 14), (19, 2, 14)]
0.1538           | [(15, 2, 13), (17, 2, 13), (19, 2, 13)]
0.1667           | [(13, 2, 12), (15, 2, 12), (17, 2, 12), (19, 2, 12), (19, 3, 18)]
0.1818           | [(13, 2, 11), (15, 2, 11), (17, 2, 11), (19, 2, 11)]
0.1875           | [(17, 3, 16), (19, 3, 16)]
0.2              | [(11, 2, 10), (13, 2, 10), (15, 2, 10), (17, 2, 10), (19, 2, 10)]
0.2143           | [(15, 3, 14), (17, 3, 14), (19, 3, 14)]
0.2222           | [(11, 2, 9), (13, 2, 9), (15, 2, 9), (17, 2, 9), (19, 2, 9)]
0.2308           | [(15, 3, 13), (17, 3, 13), (19, 3, 13)]
0.25             | [(9, 2, 8), (11, 2, 8), (13, 2, 8), (15, 2, 8), (15, 3, 12), (17, 2, 8), (17, 3,
12), (19, 2, 8)]
0.2727           | [(13, 3, 11), (15, 3, 11), (17, 3, 11), (19, 3, 11)]
0.2857           | [(9, 2, 7), (11, 2, 7), (13, 2, 7), (15, 2, 7), (17, 2, 7), (19, 2, 7)]
0.3              | [(15, 3, 10), (17, 3, 10), (19, 3, 10)]
0.3125           | [(17, 5, 16), (19, 5, 16)]
0.3333           | [(7, 2, 6), (9, 2, 6), (11, 2, 6), (13, 2, 6), (15, 2, 6), (15, 3, 9), (17, 2,
6), (17, 3, 9), (19, 2, 6), (19, 5, 15)]
0.375            | [(15, 3, 8), (17, 3, 8), (17, 6, 16), (19, 6, 16)]
0.4              | [(7, 2, 5), (9, 2, 5), (11, 2, 5), (13, 2, 5), (15, 2, 5), (17, 2, 5), (19, 2,
5), (19, 6, 15)]
0.4167           | [(13, 5, 12), (19, 5, 12)]
0.4286           | [(9, 3, 7), (13, 3, 7), (15, 3, 7), (17, 3, 7), (19, 3, 7)]
0.4375           | [(17, 7, 16), (19, 7, 16)]
0.5              | [(5, 2, 4), (7, 2, 4), (7, 3, 6), (9, 2, 4), (9, 3, 6), (11, 2, 4), (13, 2, 4),
(15, 2, 4), (15, 3, 6), (17, 2, 4), (17, 3, 6), (17, 8, 16), (19, 2, 4), (19, 7, 14), (19, 8, 16),
(19, 9, 18)]
0.5625           | [(17, 9, 16), (19, 9, 16)]
0.5714           | [(15, 8, 14), (17, 8, 14)]
0.5833           | [(13, 7, 12), (19, 7, 12)]
0.6              | [(7, 3, 5), (9, 3, 5), (13, 3, 5), (15, 3, 5), (17, 3, 5), (19, 3, 5), (19, 9,
15)]
0.6154           | [(15, 8, 13), (17, 8, 13)]
0.625            | [(15, 5, 8), (17, 5, 8), (17, 10, 16), (19, 10, 16)]
```

```
0.6364          | [(13, 7, 11), (19, 7, 11)]
0.6667          | [(5, 2, 3), (7, 2, 3), (9, 2, 3), (11, 2, 3), (13, 2, 3), (15, 2, 3), (15, 8, 12),
(17, 2, 3), (17, 8, 12), (19, 2, 3), (19, 10, 15), (19, 12, 18)]
0.6875          | [(17, 11, 16), (19, 11, 16)]
0.7             | [(11, 7, 10), (13, 7, 10), (19, 7, 10)]
0.7143          | [(9, 5, 7), (19, 10, 14)]
0.7273          | [(15, 8, 11), (17, 8, 11)]
0.75            | [(13, 9, 12), (15, 6, 8), (17, 6, 8), (17, 12, 16), (19, 9, 12), (19, 12, 16)]
0.7778          | [(11, 7, 9), (13, 7, 9), (19, 7, 9), (19, 14, 18)]
0.8             | [(15, 8, 10), (17, 8, 10), (19, 12, 15)]
0.8125          | [(17, 13, 16), (19, 13, 16)]
0.8182          | [(13, 9, 11), (19, 9, 11)]
0.8333          | [(7, 5, 6), (9, 5, 6), (13, 10, 12), (19, 10, 12), (19, 15, 18)]
0.8571          | [(9, 6, 7), (19, 12, 14)]
0.875           | [(15, 7, 8), (17, 7, 8), (17, 14, 16), (19, 14, 16)]
0.8889          | [(15, 8, 9), (17, 8, 9), (19, 16, 18)]
0.9             | [(11, 9, 10), (13, 9, 10), (19, 9, 10)]
0.9091          | [(13, 10, 11), (19, 10, 11)]
0.9167          | [(13, 11, 12), (19, 11, 12)]
0.9375          | [(17, 15, 16), (19, 15, 16)]
```

Addendum — Direct Zeta Zeros versus the Mathematical b-Network

This addendum incorporates the additional numerical experiments on the first four non-trivial zeta-zero ordinates and on the first 500 zeros. Its purpose is to sharpen the interpretation of the harmonic construction developed in this document, especially the distinction between the mathematically generated *b*-values and the reference values given directly by the ordinates of the non-trivial zeros of ζ.

1. Why the distinction between $b_{mathematical}$ and $b_{reference}$ matters

The central mechanism of the present approach is not merely the use of a spectral-looking parameter. It is the construction of a structured b-network from the polynomial/arcsine equation. In the algorithms described in the main document, the spectral parameter is generated as $\mathbf{b_t = (\arcsin(\text{signe}\cdot|s_2|) - 2k\pi) / \ln(2)}$, with the corresponding $s = 1/2 + i\, b_t$. The pilot implementation then evaluates the harmonic sums before and after the odd candidate, at n–1 and n+1, and retains a base pair when the real part of their product is negative. This is explicit in the algorithmic definition of S_{val} and in the selection criterion. ■filecite■turn0file0■L50-L56■ ■filecite■turn0file1■L28-L34■ ■filecite■turn0file1■L52-L57■

For clarity in the interpretation, we therefore use the following terminology in this addendum:

Notation	Meaning in the present analysis
$\mathbf{b_{mathematical}}$	The b-values generated by the mathematical construction (polynomial root, arcsine phase and integer k-network), i.e. the spectral network actually used by the algorithm.
$\mathbf{b_{reference}}$	The directly supplied ordinates γ_n of the non-trivial zeros of ζ, used here as an external/reference spectral sequence rather than as the output of the b-network construction.

> **The important point is therefore not that every b-value generated by the mathematical network automatically identifies a prime.** Rather, the strength of the construction is that the structured $b_{mathematical}$-network can produce negative before/after products — the sign event that the algorithm is designed to use as a possible primality indicator. By contrast, the direct $b_{reference}$ test reported below does not reproduce that behavior for the tested zero ordinates.

2. Direct test with the first four positive zeta-zero ordinates

For n = 5, the candidate under consideration is the odd integer 5, with the harmonic quantities evaluated before and after it (n–1 = 4 and n+1 = 6). When b is set directly to the first four positive non-trivial zero ordinates γ_1–γ_4, the real parts of both harmonic factors are negative in all four cases. Consequently, their real product is positive in every case.

b (= γn)	Re(Sf4)	Re(Sf6)	Re(product)	Sign
14.134725142	–0.859226	–0.826011	+0.723770	> 0
21.022039639	–0.880357	–0.149303	+0.160232	> 0
25.010857580	–0.859549	–0.087674	+0.078383	> 0
30.424876126	–0.799171	–0.866032	+0.270869	> 0

Thus the criterion “negative product = prime” fails systematically in these four isolated $b_{reference}$ evaluations: none of the four γ-values, taken alone, identifies 5 as prime. This is a particularly useful negative control because the test is applied to genuine zero ordinates rather than to an approximate or numerically unstable surrogate.

3. The negative ordinates –γn give exactly the same real test

The same experiment with $b = -\gamma_n$ gives exactly the same real parts and hence exactly the same real product. Only the imaginary parts change sign. This follows from complex conjugation: for real m, $m^{-(1/2-ib)}$ and $m^{-(1/2+ib)}$ are conjugates. Therefore changing b to –b preserves the real part of each harmonic sum and reverses only its imaginary part. The eight tests $\pm\gamma_1 \ldots \pm\gamma_4$ therefore all give a positive real product.

b (= −γn)	Re(Sf4)	Re(Sf6)	Re(product)	Sign
−14.134725142	−0.859226	−0.826011	+0.723770	> 0
−21.022039639	−0.880357	−0.149303	+0.160232	> 0
−25.010857580	−0.859549	−0.087674	+0.078383	> 0
−30.424876126	−0.799171	−0.866032	+0.270869	> 0

Interpretive consequence: the sign of the product is not obtained simply by inserting a true zeta-zero ordinate into the harmonic sums. The sign behavior appears to depend on the specific phase network and its interaction with the before/after locations n−1 and n+1.

4. Extended test over the first 500 zeta zeros

The direct-reference experiment was then extended from the first four zeros to γ_1 through γ_{500}. The observed sign sequence contains 267 positive and 233 negative products (53.4% versus 46.6%). The corresponding z-score relative to a 50/50 sign split is 1.52, which is compatible with an approximately balanced random sign sequence rather than a strong systematic bias.

Measure	Observed result	Interpretation
Positive signs	267 (53.4%)	No strong positive bias
Negative signs	233 (46.6%)	No strong negative bias
Z-score vs 50/50	1.52	Compatible with balance
Runs	280	Slightly more sign changes than random expectation
Maximum run length	5	Only one run of length 5
Runs of length 1–2	236 / 280	Alternation dominates
Lag-1 same-sign rate	44.1%	Mild anti-correlation

5. Searches for structure in n

No clear exploitable relation with the index n emerges in this 500-zero reference experiment. For parity, the sign split is approximately 55/45 for even n and 52/48 for odd n. Among the 95 prime indices up to 500, the sign split is about 45/55, whereas composite indices show about 55/45. The difference is interesting descriptively but is not sufficient, on this sample alone, to establish a primality relation. The lag-1 result (44.1% same-sign consecutive pairs) is consistent with the observed slight excess of runs and points toward mild alternation rather than persistence.

> **Conclusion of the 500-zero reference test:** the sign of $\mathrm{Re}(S_{f4} \cdot S_{f6})$ behaves like a quasi-random ±1 sequence when b is taken directly from the zeta-zero ordinates. No systematic primality signal is visible either for the tested candidate 5 or for the zero index n.

6. What this adds to the main document

These negative controls strengthen, rather than weaken, the methodological distinction at the heart of the present work. The main document already defines the harmonic test as a before/after comparison at n–1 and n+1 and selects configurations when the real part of the product is negative. ■filecite■turn0file0■L19-L26■ The pilot output also shows that, under the constructed network, negative-product pairs are in fact found for both prime and composite odd candidates, and that recurring k_a/k_b ratios can be examined across n. ■filecite■turn0file0■L284-L313■

> The new evidence suggests a more precise formulation of the strength of the approach: **$b_{\text{mathematical}}$ is not merely a substitute for the true zeta-zero ordinates; it is an engineered phase network whose interaction with the before/after evaluations can generate negative products.** That property is precisely what is required for the proposed sign-based mechanism to have any chance of flagging a candidate prime. Direct $b_{\text{reference}}$ values, by contrast, do not reproduce this behavior in the tests reported here.

This should not be read as a proof that the $b_{\text{mathematical}}$-network encodes primality. The broader results in the main document explicitly call for refining the selection criteria so as to distinguish genuine primality signatures from interference noise. ■filecite■turn0file0■L273-L282■ The present experiments sharpen that research question: the relevant object to investigate is the structured mathematical b-network and its before/after phase geometry, not the isolated insertion of γ_n values.

7. Recommended wording for the central claim

A concise formulation suitable for the paper is: *“The distinctive feature of the proposed construction is the mathematically generated b-network. Unlike direct reference values taken from the ordinates of the non-trivial zeros of ζ, these $b_{\text{mathematical}}$ values can produce negative real before/after products when the harmonic sums are evaluated*

around an odd candidate. This makes the sign criterion structurally available as a potential primality indicator, although the present evidence does not yet establish that the indicator is universally or uniquely tied to primality."